Knowledge-Augmented Dialogue over Airbus and Boeing Documents: An Empirical Comparison of RAG, GraphRAG and LightRAG

HW2 – Task One: RAG-based Dialogue System

UMAIR MUHAMMAD[†], Northwestern Polytechnical University, China

Aircraft manufacturers publish long technical manuals whose facts are difficult to look up quickly. Large language models (LLMs) can answer questions fluently, but on their own they cannot be trusted to reproduce exact figures from such documents. This report describes the design, implementation and evaluation of a retrieval-augmented dialogue system built over 6 official documents published by Airbus and Boeing (417 pages, 49,865 words). Three knowledge-augmentation methods are implemented on exactly the same text, chunks, language model and embedding model: plain retrieval-augmented generation (RAG), Microsoft GraphRAG and HKUDS LightRAG. A Gradio chat application on top of them rewrites follow-up questions into standalone questions and lets the user switch between the methods. Every model call of every method is routed through a measuring gateway, so indexing cost, query cost and latency are recorded in the same way for all systems. The methods are compared on 30 questions with reference answers, covering factual, multi-hop, global and unanswerable questions. The whole system runs on an ordinary personal computer with free LLM APIs (GLM-4.7-Flash and, for the later phases, further free GLM-Flash models) and a locally hosted embedding model (bge-m3). GraphRAG local search obtained the best mean judge score (4.50 of 5), closely followed by LightRAG (4.40), with plain RAG at 4.00 and GraphRAG global search at 3.87. The graph methods paid for this with a much more expensive index: GraphRAG needed 742 LLM calls and about 1.8 million tokens to build its index, LightRAG 115 calls and about 0.5 million tokens, and plain RAG none. At query time GraphRAG global search used about 107,000 tokens per question, seventeen times as many as plain RAG. The report also documents in detail the practical problems met when running graph-based RAG on a free API tier, and what was learned from them.

Additional Key Words and Phrases: Retrieval-Augmented Generation, GraphRAG, LightRAG, Knowledge Graphs, Dialogue Systems, Aviation Documents, Efficiency

1 Introduction

1.1 Motivation

Aircraft manufacturers publish documents for airlines, airports and maintenance organisations: *aircraft characteristics* manuals with design weights, dimensions, performance and servicing data; safety magazines describing operational lessons; and accident statistics. An airport planner who wants to know the maximum takeoff weight of a Boeing 787-9, or whether an A320neo and a 737 MAX fall into the same ICAO aerodrome code, has to search hundreds of pages across documents from different manufacturers. A conversational assistant that answers such questions correctly, and shows where each answer comes from, would save considerable time.

Large language models are a natural interface for such an assistant, but they have three well-known weaknesses for this task. First, their knowledge is stored implicitly in model weights, so they may *hallucinate* plausible but wrong numbers. Second, they do not know documents that were not part of their training

[†]Student ID: 2024380178

Author's Contact Information: Umair Muhammad, School of Computer Science, Northwestern Polytechnical University, Xi'an, China.

data or that were revised after it. Third, they cannot cite the source of an answer. Retrieval-augmented generation (RAG) [12] addresses these weaknesses by retrieving relevant passages from a document collection and giving them to the model together with the question.

1.2 From plain RAG to graph-based RAG

Plain RAG retrieves text chunks that are similar to the question. It works well when the answer is written in one place, but it struggles with two kinds of questions. *Multi-hop* questions need facts from several places, for example a comparison between an Airbus and a Boeing aircraft. *Global* questions ask about the collection as a whole, for example the main safety themes discussed in a series of magazines; no single chunk contains such an answer. Graph-based methods first turn the text into a knowledge graph of entities and relations. GraphRAG [7] additionally clusters the graph into communities and summarises them in advance, while LightRAG [9] retrieves entities and relations at two levels of abstraction with a lighter index.

1.3 Task and contributions

The homework asks to (1) collect manuals or documents from Airbus and Boeing, (2) implement RAG, GraphRAG and LightRAG for knowledge augmentation, using the GraphRAG reference page [1] and the LightRAG repository [10], and (3) build a RAG-based dialogue system. This project delivers:

- a corpus of six official Airbus and Boeing documents, converted to clean text in which tables keep their row and column structure;
- three knowledge-augmentation engines built on identical chunks, the same LLM and the same embedding model, so that differences can be attributed to the retrieval method;
- a dialogue system with follow-up question rewriting, source display and an engine selector;
- a measurement gateway and an evaluation protocol that compare indexing cost, query cost, latency and answer quality on a fixed question set;
- a detailed account of the engineering problems encountered and the lessons learned (Chapter 6).

1.4 Problem statement and objectives

1.4.1 Problem definition. Let $\mathcal{D}$ be a collection of documents from Airbus and Boeing and let a conversation consist of user messages and system answers $(u_1, a_1, u_2, a_2, \ldots, u_t)$. For the latest message u_t the system must produce an answer a_t that is

(1) **grounded**: supported by passages of $\mathcal{D}$;
(2) **correct**: consistent with what the documents state, including numbers and units;
(3) **contextual**: correct with respect to the conversation, e.g. when u_t is "And the 787-9?";
(4) **honest**: if $\mathcal{D}$ does not contain the answer, the system says so instead of guessing.

1.4.2 Question types. Four types of questions are distinguished, because the three methods are expected to differ on them:

Factual one fact stated in one place, e.g. the maximum design takeoff weight of the 787-8;

Multi-hop two or more facts that must be combined, often from an Airbus and a Boeing document, e.g. which of the 737-10 and the 787-8 has the larger lower-deck cargo volume;

Global questions about the collection as a whole, e.g. the main safety themes of the Airbus magazines;

Unanswerable questions whose answer is not in the documents, e.g. the list price of an A320neo.

1.4.3 Objectives. The objectives of the project are:

- O1 Build a document collection from official Airbus and Boeing sources and prepare it for retrieval.
- O2 Implement plain RAG, GraphRAG and LightRAG on the same collection with the same models.
- O3 Build a dialogue system that supports multi-turn conversations and any of the three engines.
- O4 Measure and compare the efficiency (indexing cost, query cost, latency) and the answer quality of the engines.
- O5 Achieve all of the above with free resources on a standard personal computer, and document what this requires.

1.5 Structure of the report

Chapter 2 explains the background of the three methods. Chapter 3 describes how the documents were collected and prepared, and Chapter 4 the design and implementation of the system. Chapter 5 describes the experimental setup and presents the results. Chapter 6 reports the observations and lessons learned while building and running the system, and Chapter 7 discusses the results and limitations and concludes the report.

2 Background and Related Work

2.1 Text embeddings and similarity search

An embedding model maps a text to a vector so that texts with similar meaning are close to each other [14]. Such models are trained contrastively on pairs of texts that should or should not match. This project uses bge-m3 [6], a multilingual model that produces 1024-dimensional vectors and accepts inputs of up to 8192 tokens. The similarity of a question vector $\mathbf{q}$ and a chunk vector $\mathbf{c}$ is measured by the cosine of their angle,

$$\cos(\mathbf{q}, \mathbf{c}) = \frac{\mathbf{q} \cdot \mathbf{c}}{\|\mathbf{q}\| \, \|\mathbf{c}\|}. \tag{1}$$

If all vectors are first normalised to unit length, the cosine equals the inner product, which allows fast exact search with libraries such as FAISS [11].

2.2 Retrieval-augmented generation

RAG [12] combines a retriever with a generator. In the *indexing phase* the documents are split into chunks and each chunk is embedded and stored. In the *query phase* the question is embedded, the k most similar chunks are retrieved and the LLM generates an answer conditioned on them. The prompt usually instructs the model to answer only from the given context and to say when the context is insufficient. The quality of plain RAG depends strongly on the chunking (chunks that are too small lose context; chunks that are too large blur the vector) and on whether the right chunk is among the top k.

2.3 Knowledge graphs and GraphRAG

A knowledge graph represents facts as nodes (entities such as *A320*, *CFM International*, *ICAO code C*) and edges (relations such as *A320 – offers engine – LEAP-1A*). GraphRAG [7, 13] builds such a graph automatically with an LLM. Its indexing pipeline (i) splits the text into text units, (ii) asks the LLM to

extract entities and relations with descriptions from every unit, (iii) merges duplicates and summarises their descriptions, (iv) partitions the graph into a hierarchy of communities with the Leiden algorithm [15], and (v) asks the LLM to write a report for every community. At query time, *local search* collects the entities closest to the question together with their neighbours, relations, source text and community reports, while *global search* answers collection-level questions by map-reduce: batches of community reports are each turned into partial answers with a relevance score (map), and the best partial answers are combined into the final answer (reduce). The price of this design is a large number of LLM calls during indexing.

2.4 LightRAG

LightRAG [9, 10] also extracts an entity–relation graph with an LLM, but it does not build communities or community reports. Each entity and each relation is stored with a description and keywords and is embedded. At query time the LLM extracts *low-level* keywords (specific entities), matched against entity vectors, and *high-level* keywords (themes), matched against relation vectors. The matched items and their one-hop neighbours, together with their source chunks, form the context. This *dual-level retrieval* is offered in the modes `local`, `global`, `hybrid` (both levels) and `mix` (graph plus vector retrieval of chunks). Because new documents can be merged into the existing graph, LightRAG also supports incremental updates.

2.5 Evaluating RAG systems

RAG systems can fail at retrieval or at generation, so evaluations commonly consider context relevance, faithfulness of the answer to the context and correctness with respect to a reference [8]. Scoring with a strong LLM ("LLM-as-a-judge") scales well but has known biases, for example towards longer answers [16]. The GraphRAG and LightRAG papers both use LLM judges to compare answers on criteria such as comprehensiveness and diversity. This project scores answers against reference answers and additionally records the computational cost of every method.

3 Data Collection and Pre-processing

3.1 Selection of documents

Following the homework, only documents published by the two manufacturers themselves were used (Table 1). The selection was made so that it is balanced between Airbus and Boeing and between technical data and safety content, which makes comparison questions across manufacturers possible:

- the Airbus A320 *Aircraft Characteristics – Airport and Maintenance Planning* manual [2];
- the Boeing 737 MAX and 787 *Airplane Characteristics for Airport Planning* manuals [4];
- two issues (23 and 25) of the Airbus *Safety First* magazine [3];
- the Boeing *Statistical Summary of Commercial Jet Airplane Accidents*, 1959–2025 [5].

Two other Airbus documents that were planned (a compilation of Flight Operations Briefing Notes and an earlier Safety First issue) could not be downloaded from the computer used for the project, because the hosting website did not load from the local network. They were replaced by Safety First issues 23 and 25 from a mirror that worked.

The three planning manuals are long (199 to 427 pages) and a large part of them consists of drawings and charts whose text is not useful for retrieval. Only the text-rich chapters were kept: the scope and

Table 1. The knowledge base: official Airbus and Boeing documents.

Document	Maker	Pages used	Words
A320 Aircraft Characteristics - Airport and Maintenance Planning (Rev. Jun 2024)	Airbus	151	13,814
Safety First magazine, issue 23	Airbus	44	11,179
Safety First magazine, issue 25	Airbus	43	11,365
737 MAX Airplane Characteristics for Airport Planning (D6-38A004, Rev. K)	Boeing	89	4,637
787 Airplane Characteristics for Airport Planning (D6-58333, Rev. Q)	Boeing	57	2,720
Statistical Summary of Commercial Jet Airplane Accidents	Boeing	33	6,150
Total		417	49,865

introduction, the aircraft description (design weights, dimensions, seating, cargo, fuel) and the aircraft performance chapter. Front matter, such as the 24-page list of effective pages of the A320 manual, was skipped. The magazines and the accident statistics were used completely.

3.2 Text extraction

The PDFs are read page by page with PyMuPDF. A first version extracted plain text only. Inspection showed that this destroys tables: in the 787 manual, each cell of the general-characteristics table was printed on its own line, so the label "MAX DESIGN TAKEOFF WEIGHT" and its value "502,500" ended up several lines apart and could fall into different chunks. The final version detects tables with PyMuPDF's table finder and writes them as Markdown rows, for example `|MAX DESIGN TAKEOFF WEIGHT|POUNDS|502,500|`, so that label, unit and value stay together. Text outside tables is kept as blocks in reading order.

3.3 Cleaning

Four cleaning steps proved necessary:

(1) **Running headers and footers.** Lines that appear on more than a third of the pages of a document (such as the document number and revision printed on every page) are removed. Table rows are exempt, because the same row label legitimately appears in many tables.
(2) **Page numbers.** Lines that contain only a page number (e.g. "2-1-1" or "12") are removed.
(3) **Hyphenation.** Words split across lines ("air- plane") are re-joined.
(4) **Broken ligatures.** The PDF of Safety First issue 25 encodes the letter pairs "fi" and "fl" as ligatures that extract as "fi rst" or "Offi cer". Unicode normalisation (NFKC) and a targeted rule repair these, which matters because a word such as "fi rst" would otherwise never match the query word "first".

Each output file starts with three lines giving the document title, the manufacturer and an identifier, so that every chunk can be traced back to its source document.

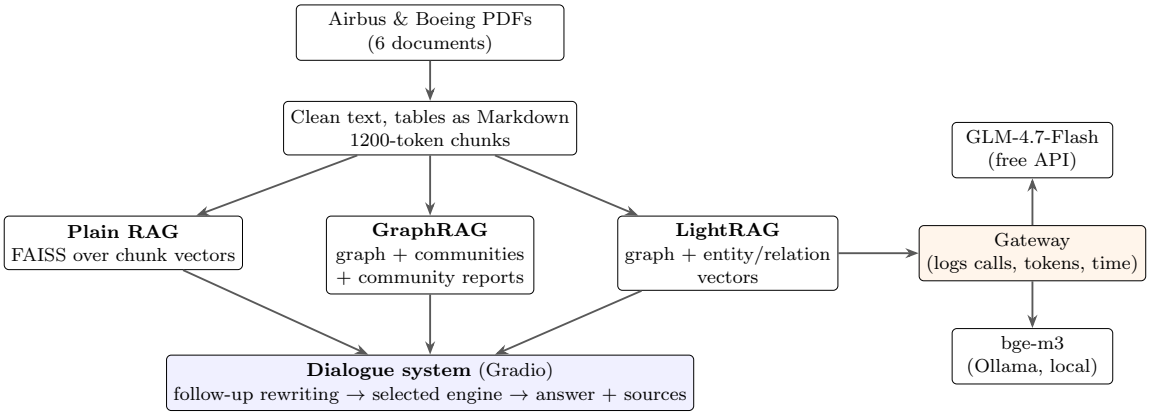


Fig. 1. System architecture. The three engines share the text, the chunks and the models; every model call of every engine passes through the measuring gateway (only one arrow is drawn).

4 System Design and Implementation

4.1 Overall architecture

Figure 1 shows the architecture. The prepared text is indexed three times, once by each method. The dialogue system sends every user question to the engine selected by the user. All LLM and embedding calls go through a local gateway.

4.2 Models and shared settings

To make the comparison fair, the three engines share everything except the retrieval method:

- **LLM**: Zhipu GLM-4.7-Flash [17], used through its free API in its OpenAI-compatible format, with the "thinking" (reasoning) mode switched off to obtain direct answers. The graph extraction of GraphRAG was done entirely with this model. For the later, shorter calls (description summaries, community reports, LightRAG indexing and the evaluation) the gateway additionally used the free GLM-4-Flash model of the same provider and account, sending each request to whichever of the two models had a free slot (Section 6.9). Both models are used in the same way by all engines, and the model that served each call is recorded in the log.
- **Embeddings**: bge-m3 served locally by Ollama on the computer's CPU.
- **Chunking**: 1200 tokens with 100 tokens of overlap, counted with the `cl100k_base` tokenizer. These are GraphRAG's and LightRAG's default chunk sizes; plain RAG was configured to match them. With this setting the corpus yields 98 chunks.
- **Extraction passes**: one extraction pass per chunk in both graph systems (`max_gleanings` = 0 in GraphRAG and `entity_extract_max_gleaning` = 0 in LightRAG).

4.3 The measuring gateway

GraphRAG and LightRAG call the LLM internally and report token usage in different ways, or not at all. To measure all engines identically, a small HTTP gateway was written with FastAPI. It offers an

OpenAI-compatible interface at `http://127.0.0.1:8800/<tag>/v1`, where `<tag>` names the caller (for example `graphrag_index` or `lightrag_query`). For each call it forwards the request to the LLM or to Ollama and writes one log line with the tag, the HTTP status, the number of attempts, the duration and the prompt and completion tokens. All results in Section 5.5 are computed from this log. Over the course of the project the gateway gained further responsibilities that turned out to be necessary on a free API tier (Chapter 6):

- it limits the number of simultaneous requests per model, because the free tier accepts only one request at a time for GLM-4.7-Flash (and two for GLM-4-Flash), and hands each request to a free model slot;
- it retries rate-limited requests (HTTP 429) with exponential back-off of up to 90 seconds;
- it aborts and repeats a request that has not been answered after 150 seconds;
- it caps every answer at 4096 output tokens;
- it converts GraphRAG's JSON-schema output requests into the JSON mode supported by the API and adds the schema to the prompt;
- it sends local traffic (to Ollama) directly and internet traffic through the system proxy;
- it merges short GraphRAG description lists locally instead of asking the LLM (Section 6.8).

4.4 Plain RAG

The chunks are embedded with bge-m3 in batches of 16, normalised to unit length and stored in an exact FAISS inner-product index. For a question, the top $k = 5$ chunks are retrieved and the LLM is prompted with the instruction to answer only from the context, to give numbers with their units, to name the source document and to say that the answer could not be found when the context does not contain it. Building this index requires no LLM call.

4.5 GraphRAG

GraphRAG is the official Microsoft implementation (Python package `graphrag`, version 3.3), which is the system documented by the reference page given in the homework [1]. The project is created with `graphrag init`, after which the generated `settings.yaml` is adapted in a few places: the model endpoints point to the gateway, the chunking and tokenizer match the other engines, the entity types are adapted to the domain (`aircraft_model`, `organization`, `component`, `system`, `airport`, `regulation`, `measurement`, `event`), one extraction pass is used, the number of concurrent requests is reduced for the free tier, and the graph is saved in GraphML format for visualisation. Indexing runs the standard pipeline (`graphrag index`). Questions are answered with `graphrag query` in *local* and in *global* mode. Community level 1 of the hierarchy is used, which keeps the number of map calls of global search small. The changed part of the configuration is shown below.

```
completion_models:
  default_completion_model:
    model_provider: openai # any OpenAI-compatible API
    model: glm-4.7-flash
    api_base: http://127.0.0.1:8800/graphrag_index/v1 # measuring gateway
chunking:
  type: tokens
  size: 1200
```

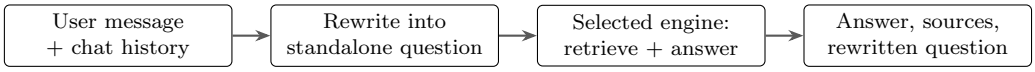


Fig. 2. One turn of the dialogue system.

```
  overlap: 100
  encoding_model: cl100k_base
extract_graph:
  entity_types: [aircraft_model, organization, component, system,
                 airport, regulation, measurement, event]
  max_gleanings: 0
concurrent_requests: 2
snapshots:
  graphml: true
```

4.6 LightRAG

LightRAG is used from the HKUDS repository given in the homework (package `lightrag-hku`, version 1.5.7). The setup follows the repository's example `examples/lightrag_openai_compatible_demo.py`, which connects LightRAG to an OpenAI-compatible LLM and an embedding function; here both point to the gateway. The documents are inserted with `ainsert`, and questions are answered with `aquery` in the `hybrid` mode (and `mix` in the chat application). The LLM response cache is disabled for queries, so that measured query times are not shortened by cached answers, but enabled for extraction, so that an interrupted indexing run can resume.

4.7 Dialogue system

The dialogue system is a Gradio chat application (Figure 2). For every user message it:

(1) takes the last six messages of the conversation;
(2) asks the LLM to rewrite the latest message into a standalone question, resolving references such as "it", "its" or "the other one" (skipped for the first message);
(3) sends the standalone question to the engine chosen in a drop-down menu (Plain RAG, GraphRAG local or global, LightRAG hybrid or mix);
(4) shows the answer together with the rewritten question, the sources and the response time.

Showing the rewritten question makes the dialogue state visible: the user can see immediately whether a follow-up was understood correctly.

4.8 Implementation overview

The project is organised as numbered scripts that can be run one by one or all together (Table 2). A PowerShell script runs the steps in order, skips steps whose result already exists, retries a failed step up to three times and restarts the running step when a restart flag file appears, which allowed bug fixes to be loaded during long runs.

The complete pipeline is reproduced with three commands:

```
powershell -ExecutionPolicy Bypass -File setup.ps1 # environment and packages
powershell -ExecutionPolicy Bypass -File run_all.ps1 # data, three indexes, evaluation, report assets
```

Table 2. Main files of the implementation.

File	Purpose
`01_prepare_data.py`	PDF to clean text (tables as Markdown, header/footer removal)
`02_build_rag.py`	chunking, bge-m3 embeddings, FAISS index
`03_build_graphrag.py`	`graphrag init` / `index` with the adapted settings
`04_build_lightrag.py`	LightRAG insertion of all documents
`engines.py`	the query engines behind one interface, follow-up rewriting
`app.py`	Gradio dialogue system
`05_evaluate.py`	30 questions × 4 engines, LLM-judge scores, dialogue test
`06_make_report_assets.py`	tables, figures and numbers used in this report
`gateway.py`, `config.py`	model access, measurement, rate-limit handling
`run_all.ps1`	runs all steps with skip, retry and restart logic

```
.\.venv\Scripts\python.exe app.py # dialogue system at http://127.0.0.1:7860
.\.venv\Scripts\python.exe show_graph.py # interactive knowledge-graph views
```

4.9 Availability of code, data and results

The complete source code, the cleaned text of the six documents, the question set, every answer and judge score, the gateway log of all model calls and this report are published at

https://github.com/umairmuhammad-hub/airbus-boeing-rag-dialogue

The pre-built indexes of the three systems (plain RAG, GraphRAG and LightRAG) are attached to the release `v1.0` of the repository as `prebuilt_indexes.zip`, so the dialogue system can be started and tested without repeating the several hours of indexing. To run it, Python 3.12, Ollama with the bge-m3 model and a free Zhipu API key are required; the repository's README describes the steps. The original PDF documents are not redistributed; they are published by Airbus and Boeing.

5 Experimental Setup and Results

5.1 Hardware and software

All experiments ran on a Windows desktop computer without a GPU, using Python 3.12 in a virtual environment. The embedding model ran on the CPU through Ollama. The LLM was the free tier of Zhipu GLM-4.7-Flash, together with the free GLM-4-Flash for the later phases, accessed over the internet. The main libraries were `graphrag` 3.3.0, `lightrag-hku` 1.5.7, `faiss-cpu`, `pymupdf`, `gradio` and `fastapi`.

5.2 Question set

30 questions were written directly from the documents, each with a reference answer and its source (Table 3): 15 factual, 7 multi-hop, 5 global and 3 unanswerable questions. Reference values were copied from the tables of the manuals; for example, the maximum design takeoff weight of the 787-8 is 502,500 lb (227,930 kg), and the A320neo is classified as ICAO aerodrome reference code 3C. Several multi-hop questions combine an Airbus and a Boeing document, and the global questions require information from several documents.

5.3 Engines evaluated

Four engines are evaluated: Plain RAG, GraphRAG local search, GraphRAG global search and LightRAG hybrid. LightRAG's `mix` mode is available in the dialogue system but was not evaluated, to save calls on the free API.

5.4 Procedure and metrics

The four engines are tested at the same time, one worker per engine, and each worker sends its questions one at a time. Every engine uses its own gateway tag (`rag_query`, `graphrag_query_local`, `graphrag_query_global`, `lightrag_query`), and the judge uses a separate tag, so all log lines of an engine's tag that fall inside a question's time window belong to that question. Running the engines in parallel means that they compete for the same free model slots; latencies therefore include waiting time and are compared with this in mind, while call and token counts are not affected. For each answer the following are recorded:

- **latency**: wall-clock time from question to answer;
- **LLM calls and tokens per question**: from the gateway log;
- **quality**: a score from 1 to 5 given by an LLM judge that compares the answer with the reference answer (5 = correct and complete, 3 = partly correct, 1 = wrong or "not found" although the answer exists). For unanswerable questions a refusal scores 5 and an invented answer scores 1.

For indexing, the wall-clock time, the number of LLM calls and the prompt and completion tokens recorded under the engine's indexing tag are reported, together with the size of the resulting index. In addition, two scripted multi-turn conversations test whether follow-up questions are rewritten correctly.

5.5 Results overview

The following subsections report the measured indexing cost, answer quality, query efficiency and the resulting knowledge graphs, and then show the dialogue system in use with example conversations and answers.

5.6 Indexing cost

Table 4 and Figure 3 show that the three systems differ by orders of magnitude in what it costs to build them. Plain RAG needs no LLM at all; its only cost is embedding the 98 chunks, which took 16.1 minutes on the CPU. LightRAG used 115 LLM calls and 508,054 tokens to extract 1,471 entities and 1,392 relations. GraphRAG used 742 calls and 1,798,394 tokens, 3.5 times as many tokens as LightRAG, for 1,416 entities, 2,658 relations and 261 community reports. Its cost is spread over three phases: one extraction call per chunk (about 100 calls with long outputs of typically 1,000–4,000 tokens), the merging of descriptions of entities found in several chunks, and the generation of community reports at four levels of the community hierarchy (20, 125, 107 and 9 communities from the coarsest to the finest level). The community reports alone are a cost that LightRAG does not have at all.

The "model time" column adds up the duration of every model call recorded by the gateway, including waiting in the free tier's queue and retries; it is therefore an upper bound of the pure computation time, but it is measured in the same way for all systems. GraphRAG's 618 minutes of model time are dominated by the free tier's waiting times during extraction and description merging (Chapter 6); LightRAG's 175 minutes

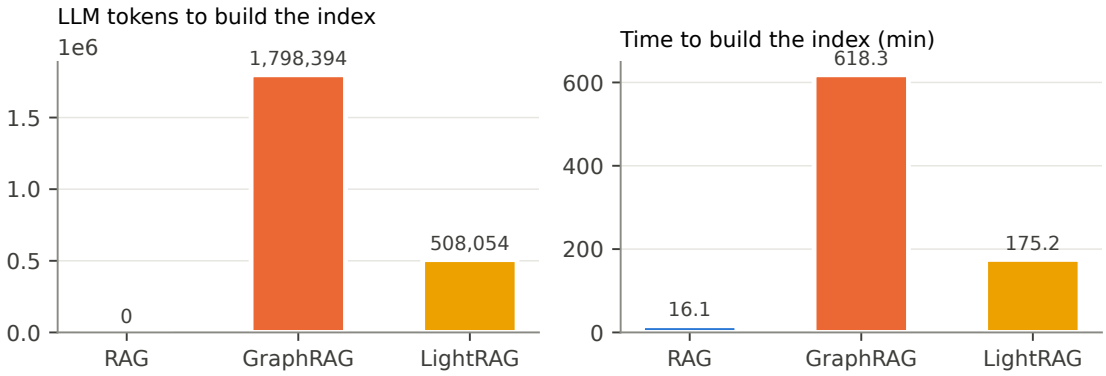


Fig. 3. Cost of building each index on the same corpus.

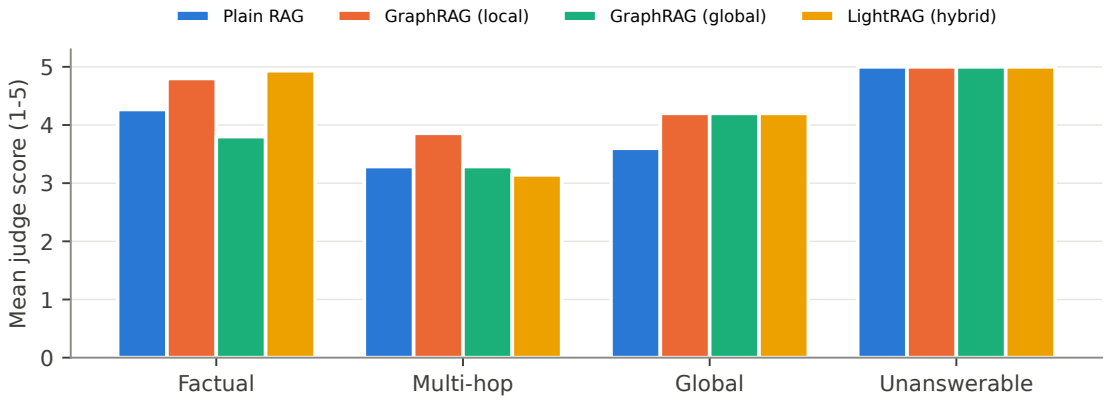


Fig. 4. Mean judge score by question type and engine.

include a first build that failed and whose cached extraction answers were reused in the second build. The LightRAG index is larger on disk (41.9 MB against 10.7 MB) because it stores vector embeddings for every entity and every relation in JSON files, whereas GraphRAG embeds entity descriptions, community reports and text units and stores them in a compact columnar database. Notably, both graph systems extracted a similar number of entities from the same text, which shows that the difference in cost comes from what is done with the graph afterwards, not from the extraction itself.

5.7 Answer quality

The overall ranking is GraphRAG local (4.50), LightRAG hybrid (4.40), plain RAG (4.00) and GraphRAG global (3.87) (Table 5, Figure 4). The share of answers scored 4 or 5 tells a similar story: 87% for LightRAG, 83% for GraphRAG local, 70% for plain RAG and 67% for GraphRAG global. The question types show where the differences come from:

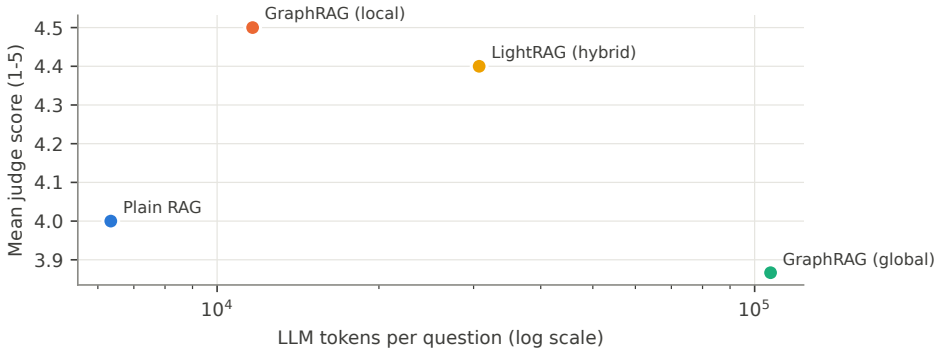


Fig. 5. Average LLM tokens spent per question against mean answer score.

- **Factual questions.** LightRAG (4.93) and GraphRAG local (4.80) answered almost all factual questions correctly, plain RAG scored 4.27. Plain RAG's failures were retrieval failures in large tables: for the seating capacity of the 737-10 it reported 440 seats instead of 204 and 230, and for the A320neo fuel capacity it took a different, larger value from the fuel table (32,168 instead of 23,740 litres). The graph methods had linked the numbers to the right aircraft model during extraction.
- **Multi-hop questions** were the hardest type for every engine (3.14–3.86). GraphRAG local was best. All four engines failed the question about the difference between the highest takeoff weights of the 787-10 and the 787-9: each found a weight variant, but not the highest one of each model, which requires reading a whole table. LightRAG made the most serious multi-hop error, stating that the 737-10 has a larger cargo volume than the 787-8, because it attached a wrong volume to the 737-10.
- **Global questions.** Both graph methods (4.20) were better than plain RAG (3.60), which can only see five chunks. GraphRAG global search did not outperform local search on this small corpus.
- **Unanswerable questions.** All engines correctly said that the list price of an A320neo, the number of A350 deliveries in 2025 and the wingspan of the 777X are not in the documents (5.00 each). The grounding instruction in the prompts was therefore effective, and no engine invented an answer.

GraphRAG global search was the weakest engine on factual questions (3.80). It answers from community reports, which summarise groups of entities and lose exact figures; several of its answers were long, well-structured reports that did not contain the requested number, for example the maximum takeoff weight of the 787-8.

5.8 Query efficiency

Table 6 and Figure 5 compare the cost of answering. Plain RAG and GraphRAG local search need one LLM call per question, LightRAG two (one to extract keywords from the question and one to answer), and GraphRAG global search on average 8.9 calls, one "map" call per batch of community reports plus a final "reduce" call. In tokens per question, plain RAG is the cheapest (6,342), followed by GraphRAG local (11,643), LightRAG (30,724) and GraphRAG global (107,064). GraphRAG local search therefore gave the



Fig. 6. The 45 best-connected entities of the GraphRAG graph (left) and the LightRAG graph (right).

best quality at the second-lowest cost, whereas global search was both the most expensive and the least accurate method on this set.

The latencies must be read with care. The four engines were evaluated at the same time and shared six slots on four free models, and LightRAG answered its questions last, after the other engines had finished, when no other engine was competing for the slots. Its median latency of 10.1 seconds is therefore not directly comparable with the 57.5 seconds of GraphRAG local search, part of which is waiting time. Two observations do not depend on the load: GraphRAG starts a new command-line process for every question, which loads the index files each time and adds several seconds; and global search with about nine calls is inherently the slowest method. Token counts, unlike latencies, are independent of the load and are the more reliable efficiency measure here.

5.9 The knowledge graphs

The two graphs were extracted from the same chunks but look different (Table 7, Figure 6). GraphRAG's most connected entities are general concepts: AIRBUS (degree 111), A320-200, A320NEO, AIRCRAFT, FATAL ACCIDENTS and BOEING. GraphRAG writes entity names in capitals and merges every mention of a name into one node, so general entities collect many relations. LightRAG keeps names as written and produced more specific hubs such as "Model 787-10", "A/C CONFIGURATION" and "Worldwide Commercial Jet Fleet"; its maximum degree (34) is much lower, and its graph has slightly more entities but about half as many relations (1,392 against 2,658). The graph pictures show that both graphs contain a dense cluster around the A320 family, which is described in the longest document, and separate clusters for the 787 and the 737 MAX that are only weakly connected to it. This weak connection helps to explain why comparisons between an Airbus and a Boeing aircraft remained difficult. Both graphs also contain noise typical of automatic extraction from technical documents, for example entities for the paper on which a magazine was printed or for single table headings.

5.10 The dialogue system in use

Figure 7 shows the web interface of the dialogue system during a real conversation with the Plain RAG engine. The user types a question, chooses the knowledge-augmentation engine in the drop-down list under

"Additional Inputs", and receives the answer together with a footer that shows the engine, the response time, the standalone question that was actually searched for, and the source documents of the retrieved context. The interface also offers example questions, a button to repeat or undo a turn, and a button to clear the conversation.

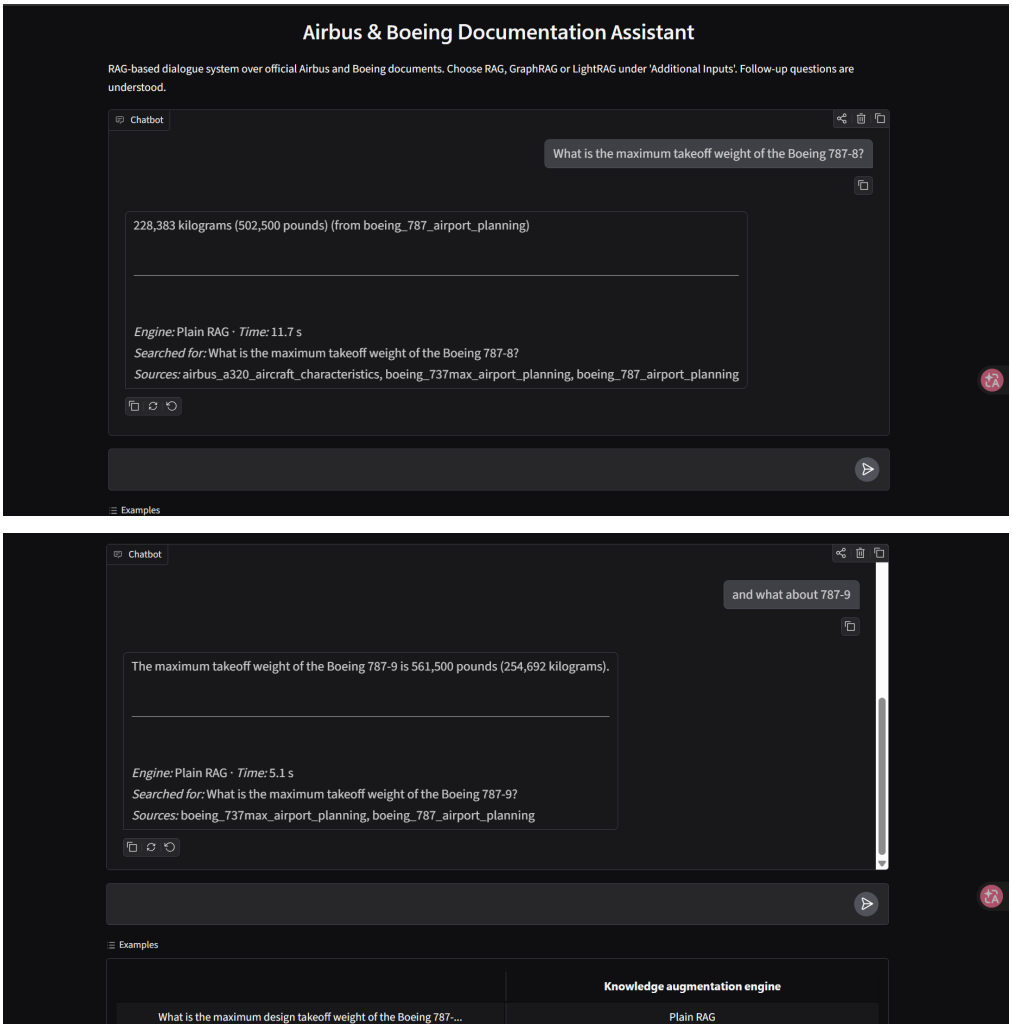


Fig. 7. The dialogue system in use with the Plain RAG engine. Top: the first question with the answer, the engine, the response time, the question that was searched for and the source documents. Bottom: the follow-up "and what about 787-9" was rewritten into the standalone question "What is the maximum takeoff weight of the Boeing 787-9?" before retrieval.

The demonstration also shows the kind of errors that remain. The first answer gives the correct 502,500 lb, but its kilogram value (228,383 kg) differs from the 227,930 kg printed in the manual, i.e. the model recomputed the conversion instead of copying it. The second answer gives 561,500 lb, which is one of several weight variants of the 787-9 listed in the manual rather than the highest one (571,500 lb). Because the

searched question and the source documents are shown under every answer, the user can check such values in the original document.

To show how the engines differ on the same input, Table 8 lists shortened answers of all four engines to two test questions, together with the judge scores.

5.11 Dialogue behaviour

Besides the interactive use, two scripted multi-turn conversations were run automatically with LightRAG hybrid as the engine (Table 9 shows the rewritten questions, Table 10 the answers).

The rewriting step worked for the typical elliptical follow-ups: “And the 787-9?” became “What is the maximum design takeoff weight of the Boeing 787-9?” and “What about the A320neo?” became “Which engine does the Airbus A320neo use?”. With the correct standalone question, the second dialogue was answered correctly. In the first dialogue, however, LightRAG answered the correctly rewritten 787-9 question with the takeoff weight of the 737-9 (194,700 lb). The names “787-9” and “737-9” differ in one character, and in the keyword and entity retrieval of LightRAG the wrong aircraft was matched; the same question with Plain RAG (Figure 7) found a 787-9 value. This error comes from retrieval, not from the dialogue handling. The third turn of the first dialogue shows a limitation of the dialogue handling itself: “Which of the two can carry more passengers in dual class?” was rewritten to “Which of the two aircraft can carry more passengers in dual class?”, so the reference “the two” was not resolved to the 787-8 and the 787-9, and the engine then compared two unrelated aircraft. The rewriting prompt sees the conversation, but a short free model sometimes keeps a reference unresolved; giving it an explicit instruction to name all entities, or passing the previous question to the retriever as well, would address this.

6 Observations and Lessons Learned

Running two graph-based RAG systems on a free API tier and a desktop CPU produced a number of problems that are not visible in the papers or the library documentation. They are reported here because each of them changed the design of the system, and because they are important for anyone who wants to reproduce such a comparison under similar constraints. Table 11 summarises the behaviour of the API and the embedding model as recorded by the gateway.

6.1 Data preparation matters more than expected

The first plain-text extraction looked acceptable at a glance but broke every table into one cell per line. Since most factual questions in this domain are answered by table rows (weights, capacities, fuel volumes), this would have made the tables nearly useless for retrieval. Keeping tables as Markdown rows was the single most important pre-processing decision. A second lesson was that figures carry information that text extraction cannot recover: the dimension drawings of the A320 manual extract as a list of numbers such as “35.80 m” without the labels that say which dimension they belong to. Questions about wingspans were therefore not used, because no system could answer them from the text. Finally, PDF encoding details (ligatures, running headers) silently change the text and must be checked by reading samples of the output.

6.2 Free API tiers: one request at a time

The free GLM-4.7-Flash tier accepts only one request at a time per account. The first indexing attempt sent two requests in parallel; almost every second request was rejected with HTTP 429 and error code 1302 ("the account has reached its rate limit"), and the retries were rejected as well, so GraphRAG appeared to be stuck at its first chunk. A test that sent five parallel requests confirmed the limit: the free GLM-4.7-Flash accepted none of them while the indexing job was occupying the single slot, and the older free GLM-4-Flash accepted two of five at about 36 output tokens per second. Serialising all requests in the gateway and retrying with longer back-off solved the problem, at the price of a much longer indexing time. The observation that the limit applies per model was used later to speed up the remaining phases (Section 6.9).

6.3 Runaway generations

GraphRAG's extraction prompt does not limit the length of the answer. On one chunk, the model started to repeat itself and produced 31,980 output tokens in 453 seconds instead of the usual 1,000–4,000 tokens. Capping every answer at 4096 tokens in the gateway removed this failure. Across the extraction calls recorded by the gateway, the median answer contained about 1,900 output tokens for about 3,000 input tokens, and roughly one call in five reached the cap, which shows that the extraction step is dominated by generated output rather than by input.

6.4 Networks, proxies and VPNs

The project computer used a VPN. Several problems were caused by the interaction between the VPN's system proxy and local services:

- requests to the local embedding server first failed with HTTP 502 because they were routed towards the proxy;
- after local traffic was excluded from the proxy, requests to the LLM API also bypassed it and took 35–55 seconds for a two-word answer, while the same request sent through the proxy took about 2 seconds;
- some API connections stalled for several minutes without an answer.

The final design sends local traffic directly, sends internet traffic explicitly through the system proxy, and aborts a request after 150 seconds without an answer. A related lesson was that a helper process started in the background by one script did not survive reliably on Windows; running the gateway inside each script's own process made the system robust. In the final version the run script itself starts the gateway as a separate program, watches it, and starts it again if it stops, so that the GraphRAG and LightRAG builds can share one gateway while they run in parallel.

6.5 Local embeddings on a CPU

Embedding the 98 chunks of about 1,200 tokens each took 16.3 minutes on the computer's CPU, i.e. about 2.5 minutes per batch of 16 chunks or roughly 10 seconds per chunk. This is acceptable for a corpus of this size but would become the bottleneck for a larger collection; a GPU or a hosted embedding API would remove it. Embedding a single question, in contrast, takes well under a second, so query latency is dominated by the LLM.

6.6 Silent failures

The first LightRAG build appeared to finish normally: the script ended without an error and wrote its statistics file. Only a look at LightRAG's document status store showed that all six documents had been marked as *failed*. While GraphRAG was building its index at the same time, the CPU-bound embedding server answered LightRAG's batches more slowly than LightRAG's internal limit of 30–60 seconds per embedding call, so LightRAG cancelled the processing of every document. Its insert function reports such failures only in the status store and does not raise an exception. Three changes followed: the embedding and LLM time limits of LightRAG were raised to 15 and 10 minutes, its embedding calls were limited to one batch at a time, and the build script now checks the status of every document and counts the build as finished only when all of them are processed. The failed build was moved aside, but its cache of 48 extraction answers was kept, so the rebuild did not repeat those LLM calls. The general lesson is that a step should be judged by checking its output, not by the absence of an error message.

6.7 Long-running jobs need resumability

Because indexing took hours, it had to survive interruptions: stopping a run to load a fix, network stalls and rate-limit errors. Three mechanisms made this possible: GraphRAG's LLM cache (finished chunks are not sent again when the index is rebuilt), LightRAG's extraction cache, and the run script, which skips finished steps, retries failed steps and restarts the running step when a flag file appears. Without them every fix would have cost hours of recomputation.

6.8 The summarisation bottleneck

After entity extraction, GraphRAG merges every entity and relationship that was found in more than one chunk by asking the LLM to summarise its list of descriptions. On this corpus that meant 4,253 summarisation items. Each request is short (about 200 input and 120 output tokens), but on the free tier each one waited in the provider's queue: the median call took about 50–65 seconds, mostly in rate-limit retries, so the summaries alone would have needed more than a day. LightRAG solves the same problem differently: it calls the LLM only when an entity has at least eight description fragments (its `FORCE_LLM_SUMMARY_ON_MERGE` default) and otherwise simply joins them. The same rule was applied to GraphRAG inside the gateway: a summarisation request with fewer than eight descriptions is answered locally by joining the distinct descriptions (at most 500 words), and only longer lists are sent to the model. This keeps the two graph systems comparable and is recorded in the log as `local_merge`. A practical lesson came with it: the fix only takes effect once the running process is restarted, so a change that is copied to the computer but not loaded has no effect at all, and checking that the deployed files really are the new versions became part of every fix.

6.9 Using two free models at once

The concurrency limit applies per model, not per account. The provider's older free model, GLM-4-Flash, accepted two simultaneous requests while GLM-4.7-Flash was busy. The gateway was therefore extended with a small pool of model slots (one for GLM-4.7-Flash, two for GLM-4-Flash): each request takes whichever slot is free, and after a rate-limit error it is retried preferably on the other model. When the four engines were evaluated in parallel this was still too few, and answers waited two to three minutes for a free slot, so

two more free models of the same provider (GLM-4.5-Flash and GLM-4-Flash) were added, giving six slots on four models. A model that the account cannot use is removed from the pool automatically. Together with building the GraphRAG and LightRAG indexes at the same time, this made it possible to finish the community reports, the LightRAG index and the whole evaluation within about three hours instead of days. In practice GLM-4.7-Flash was busy most of the time, and the GLM-4-Flash variants served the majority of the calls of the later phases.

6.10 Measuring fairly

Library-internal token counters are not comparable across systems, and wall-clock times depend on the API's current load. Logging every call at one point (the gateway) gave comparable call and token counts, while times are reported with the caveat that they depend on the free tier's load at the time of the run.

6.11 Summary of what was learned

- Retrieval quality starts with document preparation; tables and figures need special care.
- Graph-based RAG moves cost from query time to indexing time, and on a free API this indexing cost dominates the whole project.
- Free APIs impose concurrency limits and occasional failures that must be handled explicitly with serialisation, retries, time-outs and output caps.
- A measurement layer in front of all model calls is a simple way to compare different RAG libraries fairly.
- Long experiments must be resumable and observable; logging each call made it possible to diagnose every problem described in this section.

7 Discussion and Conclusion

7.1 Which method for which question?

The results support a clear practical recommendation for documents of this kind. For factual questions about specific aircraft, which make up most real questions in this domain, LightRAG and GraphRAG local search are both more accurate than plain RAG, because entity extraction links each number to the aircraft model and the table it belongs to. Plain RAG is still a strong baseline: it is the cheapest method by a wide margin, needs no LLM for indexing, and answered 70% of all questions well. GraphRAG global search, the most distinctive feature of GraphRAG, did not pay off on this corpus: it was the most expensive method and the least accurate one, and it was not better than local search even on global questions. It is designed for questions about large collections where the answer is spread over many documents; with six documents, local search already sees enough context.

Between the two graph systems the main difference is cost. LightRAG reached almost the same quality as GraphRAG local search (4.40 against 4.50, a difference of one question-score point in thirty questions) for less than a third of the indexing tokens, with no community reports to generate, and with the possibility to add documents incrementally. When the indexing budget matters, as it did on a free API, LightRAG is the better trade-off; when the best answer quality matters and the index is built once, GraphRAG local search is a good choice.

Multi-hop comparisons remained the weak point of all methods. They need two exact values from tables in different documents, and an error in either value makes the comparison wrong. None of the methods performs explicit reasoning over table rows, which suggests that structured extraction of the key tables (weights, capacities, dimensions) into a small database would help more than further improvements in retrieval.

7.2 Limitations

- **Size of the test set.** With 30 questions, small differences between engines are not statistically significant.
- **Free models.** All engines use the same two free models; a stronger model might change the ranking, especially for graph extraction, whose quality depends on the model. Because requests are assigned to whichever model is free, each engine's answers come from a mixture of the two models.
- **LLM judge.** The judge is itself an LLM and may favour longer answers [16]. Reading the answers showed individual misjudgements in both directions; for example, an answer stating that the A320neo fuel capacity is not given in the documents received a 5, although the reference value exists. Such errors affect all engines, but individual scores should not be over-interpreted.
- **Text-only knowledge.** Information contained in drawings and charts is not represented.
- **Timing.** Latencies and indexing times depend on the load of the free API during the run; token counts do not.
- **Output cap.** The 4096-token cap on answers prevents runaway generations but may truncate a few very long extraction outputs, which can remove some entities from the graphs.

7.3 Conclusion

This project built a RAG-based dialogue system over official Airbus and Boeing documents and implemented three knowledge-augmentation methods on identical data and models: plain RAG, Microsoft GraphRAG and HKUDS LightRAG. A measuring gateway made the efficiency of the three methods directly comparable, and an evaluation on factual, multi-hop, global and unanswerable questions compared their answer quality. The main findings are:

- graph-based retrieval improved answer quality over plain RAG, from 4.00 to 4.50 (GraphRAG local) and 4.40 (LightRAG), mainly on factual questions about specific aircraft and on global questions;
- the price is paid at indexing time: GraphRAG needed 1.8 million and LightRAG 0.5 million LLM tokens to build their indexes, while plain RAG needed none;
- GraphRAG global search was the most expensive and least accurate method on this small corpus;
- no engine invented answers to questions outside the documents, but all engines struggled with multi-hop comparisons that need exact values from two tables;
- on a free API tier, the engineering around the libraries (rate limits, time-outs, output caps, silent failures, resumable runs and fair measurement) took more effort than the use of the libraries themselves.

For a question-answering assistant over aircraft documentation, LightRAG offers the best balance between quality and cost, GraphRAG local search the best quality, and plain RAG remains a useful cheap baseline.

7.4 Future work

Future work includes extracting information from drawings with a vision model, adding keyword (BM25) retrieval for exact part and document numbers, evaluating with human judges and a larger question set, and repeating the comparison with a model that allows parallel requests, which would shorten graph indexing considerably.

Acknowledgments

The two knowledge-augmentation libraries used in this work are the ones referenced in the homework: Microsoft GraphRAG (https://graphrag.homes/) and HKUDS LightRAG (https://github.com/HKUDS/LightRAG). The documents are published by Airbus and Boeing.

References

[1] [n. d.]. GraphRAG: Graph-based Retrieval-Augmented Generation (reference page). https://graphrag.homes/. Accessed October 2026.

[2] Airbus. 2024. A320 Aircraft Characteristics – Airport and Maintenance Planning. https://aircraft.airbus.com/en/customer-care/fleet-wide-care/airport-operations-and-aircraft-characteristics/aircraft-characteristics.

[3] Airbus Product Safety Department. 2018. Safety First – The Airbus Safety Magazine, Issues 23 (2017) and 25 (2018). https://safetyfirst.airbus.com/.

[4] Boeing Commercial Airplanes. 2025. Airplane Characteristics for Airport Planning: 737 MAX (D6-38A004) and 787 (D6-58333). https://www.boeing.com/commercial/airports/plan-manuals.

[5] Boeing Commercial Airplanes. 2026. Statistical Summary of Commercial Jet Airplane Accidents, Worldwide Operations 1959–2025. https://www.boeing.com/content/dam/boeing/v2/safety/statsum.pdf.

[6] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv preprint arXiv:2402.03216* (2024).

[7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv preprint arXiv:2404.16130* (2024).

[8] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. RAGAS: Automated Evaluation of Retrieval Augmented Generation. *arXiv preprint arXiv:2309.15217* (2023).

[9] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Retrieval-Augmented Generation. *arXiv preprint arXiv:2410.05779* (2024).

[10] HKUDS. [n. d.]. LightRAG: Simple and Fast Retrieval-Augmented Generation. https://github.com/HKUDS/LightRAG. Python package `lightrag-hku`, version 1.5.7.

[11] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 9459–9474.

[13] Microsoft Research. [n. d.]. GraphRAG: A Modular Graph-based Retrieval-Augmented Generation System. https://github.com/microsoft/graphrag. Version 3.3.0.

[14] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of EMNLP-IJCNLP*. 3982–3992.

[15] Vincent A. Traag, Ludo Waltman, and Nees Jan van Eck. 2019. From Louvain to Leiden: Guaranteeing Well-Connected Communities. *Scientific Reports* 9, 1 (2019), 5233.

[16] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*.

[17] Zhipu AI. [n. d.]. GLM-4.7-Flash. https://docs.bigmodel.cn/cn/guide/models/free/glm-4.7-flash. Free model on the BigModel open platform.

Table 3. The evaluation questions (A: = Airbus, B: = Boeing document).

#	Type	Question	Source
1	factual	What is the ICAO Aerodrome Reference Code of the A320neo?	A:a320_AC
2	factual	When is an A320-200 classified as ICAO aerodrome code 4C instead of 3C?	A:a320_AC
3	factual	What is the standard seating capacity of the A320 given in the Airbus Aircraft Characteristics manual?	A:a320_AC
4	factual	What is the maximum total usable fuel capacity of the A320neo?	A:a320_AC
5	factual	Which two engine options are offered on the A320neo family?	A:a320_AC
6	factual	Which engine does the Boeing 737 MAX use?	B:737max_ACAP
7	factual	What is the two-class and single-class seating capacity of the Boeing 737-10?	B:737max_ACAP
8	factual	What is the usable fuel capacity of the Boeing 737 MAX without auxiliary tanks?	B:737max_ACAP
9	factual	What is the maximum design takeoff weight of the Boeing 787-8?	B:787_ACAP
10	factual	What is the dual-class seating capacity of the Boeing 787-9?	B:787_ACAP
11	factual	How does the Boeing 787 differ from earlier airplanes in its pneumatic system?	B:787_ACAP
12	factual	How many fatal accidents did the worldwide commercial jet fleet have in 2016-2025 according to Boeing's Statistical Summary?	B:statsum
13	factual	According to Boeing's Statistical Summary, what share of fatal accidents in 2016-2025 happened during landing, and how much flight time does landing represent?	B:statsum
14	factual	What is the Soft Go-Around function introduced by Airbus?	A:safety_first_23
15	factual	When did Airbus start publishing a dedicated safety magazine, and when did it become 'Safety First'?	A:safety_first_25
16	multi-hop	Which has the larger lower-deck cargo volume, the Boeing 737-10 or the Boeing 787-8, and by roughly how much?	B:737max_ACAP, B:787_ACAP
17	multi-hop	Compare the usable fuel capacity of the A320neo with that of the 737 MAX (no auxiliary tanks). Which carries more?	A:a320_AC, B:737max_ACAP
18	multi-hop	Compare the single-class seating capacity of the A320 and the Boeing 737-8.	A:a320_AC, B:737max_ACAP
19	multi-hop	What is the difference between the highest maximum design takeoff weights of the 787-10 and the 787-9?	B:787_ACAP
20	multi-hop	Are the A320neo and the 737 MAX in the same ICAO aerodrome reference code letter?	A:a320_AC, B:737max_ACAP
21	multi-hop	Which engine manufacturers power the Boeing 787 and the Airbus A320neo?	B:787_ACAP, A:a320_AC
22	multi-hop	Compare the maximum design landing weight of the Boeing 787-8 with the maximum landing weight of the A320-200.	B:787_ACAP, A:a320_AC
23	global	What are the main safety themes discussed across the Airbus Safety First magazines?	A:safety_first_23, A:safety_first_25
24	global	How do the Airbus and Boeing documents describe the overall trend in commercial aviation safety?	B:statsum, A:safety_first_23, A:safety_first_25
25	global	What kinds of information do the airport-planning manuals from Airbus and Boeing provide?	A:a320_AC, B:737max_ACAP, B:787_ACAP
26	global	Summarize the main features of the Boeing 737 MAX family described in its planning manual.	B:737max_ACAP
27	global	Which design weight terms are used by both manufacturers and what do they mean?	A:a320_AC, B:787_ACAP, B:737max_ACAP
28	unanswerable	What is the list price of a new Airbus A320neo?	–
29	unanswerable	How many A350 aircraft did Airbus deliver in 2025?	–
30	unanswerable	What is the wingspan of the Boeing 777X?	–

Table 4. Indexing cost of the three systems on the same corpus (measured by the gateway).

System	Model time (min)	LLM calls	LLM tokens	Embedded	Chunks	Entities	Relations	Comm.	Size (MB)
RAG	16.1	0	0	98	98	–	–	–	–
GraphRAG	618.3	742	1,798,394	1,782	98	1,416	2,658	261	10.7
LightRAG	175.2	115	508,054	4,131	99	1,471	1,392	–	41.9

Table 5. Answer quality (LLM-judge score 1–5, mean) by question type.

Engine	Factual	Multi-hop	Global	Unans.	All	Score≥4
Plain RAG	4.27	3.29	3.60	5.00	4.00	70%
GraphRAG (local)	4.80	3.86	4.20	5.00	4.50	83%
GraphRAG (global)	3.80	3.29	4.20	5.00	3.87	67%
LightRAG (hybrid)	4.93	3.14	4.20	5.00	4.40	87%

Table 6. Query-time efficiency over the 30 test questions.

Engine	Mean (s)	Median (s)	LLM calls/q	Tokens/q	Errors
Plain RAG	37.4	9.9	1.0	6,342	0
GraphRAG (local)	65.9	57.5	1.0	11,643	0
GraphRAG (global)	106.8	101.7	8.9	107,064	0
LightRAG (hybrid)	13.7	10.1	2.0	30,724	0

Table 7. The ten most connected entities in each knowledge graph.

#	GraphRAG entity	Degree	LightRAG entity	Degree
1	AIRBUS	111	Model 787-10	34
2	A320-200	99	A/C CONFIGURATION	31
3	A320NEO	80	Fatal accidents	31
4	AIRCRAFT	72	Airbus	29
5	FATAL ACCIDENTS	42	Model 787-8	25
6	BOEING	41	Model 787-9	25
7	A320	39	Airbus S.A.S.	24
8	LANDING GEAR	36	Worldwide Commercial Jet Fleet	24
9	A320 FAMILY	34	MEL	22
10	737 MAX	31	Maintenance	22

Table 8. Shortened answers of the four engines to two test questions (judge score in brackets).

Q7: What is the two-class and single-class seating capacity of the Boeing 737-10? *Reference: 204 and 230 seats.*	
Plain RAG (1)	"The two-class seating capacity ... is not explicitly mentioned in the provided context. However, the single-class seating capacity is 440 all-economy seats ..."
GraphRAG local (5)	"In a two-class configuration, the Model 737-10 can accommodate 204 passengers, while in a single-class configuration, it can seat up to 230 passengers."
GraphRAG global (3)	A long structured report stating that the two-class configuration "typically accommodates 110 seats", a value that does not appear in the manual.
LightRAG hybrid (5)	"The Boeing 737-10 has a two-class seating capacity of 204 passengers and a single-class seating capacity of 230 passengers."
Q16: Which has the larger lower-deck cargo volume, the 737-10 or the 787-8? *Reference: the 787-8 (4,826 vs. 1,958 cu ft).*	
Plain RAG (3)	Correct choice and 787-8 volume (4,826 cu ft), but gave compartment lengths instead of the 737-10 volume.
GraphRAG local (5)	"The 787-8 offers ... 4,826 cubic feet ... the Boeing 737-10 has a lower deck cargo volume of 1,958 cubic feet."
GraphRAG global (4)	Correct conclusion (the 787-8 is much larger) without the exact volumes.
LightRAG hybrid (1)	Wrong conclusion: attributed 6,722 cu ft to the 737-10 and called it the larger one.

Table 9. Follow-up questions and the standalone questions produced by the dialogue system.

Dlg.	Turn	User message	Rewritten standalone question
1	1	What is the maximum design takeoff weight of the Boeing 787-8?	What is the maximum design takeoff weight of the Boeing 787-8?
1	2	And the 787-9?	What is the maximum design takeoff weight of the Boeing 787-9?
1	3	Which of the two can carry more passengers in dual class?	Which of the two aircraft can carry more passengers in dual class?
2	1	Which engine does the Boeing 737 MAX use?	Which engine does the Boeing 737 MAX use?
2	2	What about the A320neo?	Which engine does the Airbus A320neo use?

Table 10. Answers of LightRAG (hybrid) in the scripted dialogues.

Dlg.	Turn	Answer (shortened)	Correct?
1	1	502,500 lb (227,930 kg)	yes
1	2	194,700 lb (88,314 kg) – the value of the 737-9, not of the 787-9	no
1	3	compared the 787-10 with the BBJ MAX 8 instead of the 787-8 and the 787-9	no
2	1	CFM LEAP-1B	yes
2	2	Pratt & Whitney PW1100G and CFM LEAP-1A	yes

Table 11. Behaviour of the free LLM API and the local embedding model, from the gateway log.

Phase	LLM calls	Retried	Median s/call	Max output tok.	Embed req.	Median s/embed
RAG indexing	0	0	–	–	7	154.3
GraphRAG indexing	745	195	28.0	31,980	138	10.6
LightRAG indexing	115	37	46.1	4,096	421	3.1
RAG queries	31	3	5.7	1,234	31	1.3
GraphRAG local queries	31	2	13.9	1,104	31	1.0
GraphRAG global queries	267	4	7.6	4,000	0	–
LightRAG queries	70	0	3.5	1,145	35	0.2
Dialogue rewriting	3	0	0.7	19	0	–
LLM judge	121	2	0.6	3	0	–